

MODULE 3 LAB WORKBOOK

Python BDD RESTful Automations

Name – Sristi Saha

Institute – Institute of Engineering & Management, Kolkata

Enrollment No. – 12023052016066

Module	Python BDD RESTful Automations
Programming Language	Python
BDD Framework	Behave
Web Automation	Selenium WebDriver
API Automation	Requests library and JSON response validation
IDE	PyCharm Community Edition
Experiments Covered	Assignment 1, Assignment 2 and Assignment 3

Experiment List

Experiment	Experiment Title	Primary Concepts
1	Selenium Python and Behave BDD	BDD setup, Gherkin feature, step definitions, browser automation and assertions
2	Data-Driven API Automation using Python Behave	Requests, GET calls, JSON parsing, Scenario Outline, Examples and response validation
3	Selenium Page Object Model in Python Behave	POM, page classes, locators, reusable methods, positive and negative login scenarios

Experiment 1: Implementation of Selenium Python and Behave BDD

Module	Python BDD RESTful Automations
Experiment Title	Implementation of Selenium Python and Behave BDD
Web Automation	https://demoqa.com/text-box

Problem Statement

Set up a Python Behavior-Driven Development (BDD) framework using Behave and Selenium WebDriver, then implement and execute an end-to-end browser automation scenario. The test should open a web form, enter user details, submit it, and validate the resulting output.

Objective

To configure a Python BDD framework in PyCharm, understand the relationship between Gherkin feature files and Python step definitions, and execute an end-to-end Selenium test with visible checkpoints for screenshot evidence.

Tools, Software, and Concepts Used

- Python 3
- Behave BDD framework
- Selenium WebDriver 4 and Google Chrome
- Gherkin syntax: Feature, Scenario, Given, When, Then
- Assertions, browser navigation, element locators and form submission
- Windows PowerShell terminal and virtual environment

Implementation Steps

1. Create a Python project and configure a virtual environment in PyCharm.
2. Install the required packages using pip install selenium behave.
3. Create the features directory, a Gherkin feature file, environment.py and a steps subdirectory.
4. Define the DemoQA Text Box scenario in Gherkin.
5. Implement the corresponding step definitions using Selenium WebDriver.

Source Code —

features/login.feature

Feature: Selenium automation using Python Behave

Scenario: Submit the DemoQA text box form

Given I open the DemoQA text box page

When I enter my name, email and address

And I submit the form

Then I should see the submitted details

features/environment.py

```
from selenium import webdriver
from selenium.webdriver.chrome.options import Options
```

```
def before_scenario(context, scenario):
    options = Options()
    options.add_argument("--start-maximized")

    context.driver = webdriver.Chrome(options=options)
    context.driver.implicitly_wait(10)
```

```
def after_scenario(context, scenario):
    if hasattr(context, "driver"):
        input("\nPress ENTER to close the browser...")
        context.driver.quit()
```

features/steps/selenium_steps.py

```
from behave import given, when, then
from selenium.webdriver.common.by import By
from selenium.webdriver.support.ui import WebDriverWait
from selenium.webdriver.support import expected_conditions as EC
```

```
@given("I open the DemoQA text box page")
def open_page(context):
    context.driver.get(
        "https://demoqa.com/text-box"
    )
    print("Opened DemoQA Text Box page")
    input("Press ENTER to continue.")
```

```
@when("I enter my name, email and address")
```

```
def enter_details(context):
    driver = context.driver
    driver.find_element(
        By.ID, "userName"
    ).send_keys("Sristi Saha")

    driver.find_element(
        By.ID, "userEmail"
    ).send_keys("sristi@example.com")
    driver.find_element(
        By.ID, "currentAddress"
    ).send_keys("Kolkata, India")
```

```
    driver.find_element(
        By.ID, "permanentAddress"
    ).send_keys("West Bengal, India")
    print("Form details entered successfully")
    input("Press ENTER to continue.")
```

```

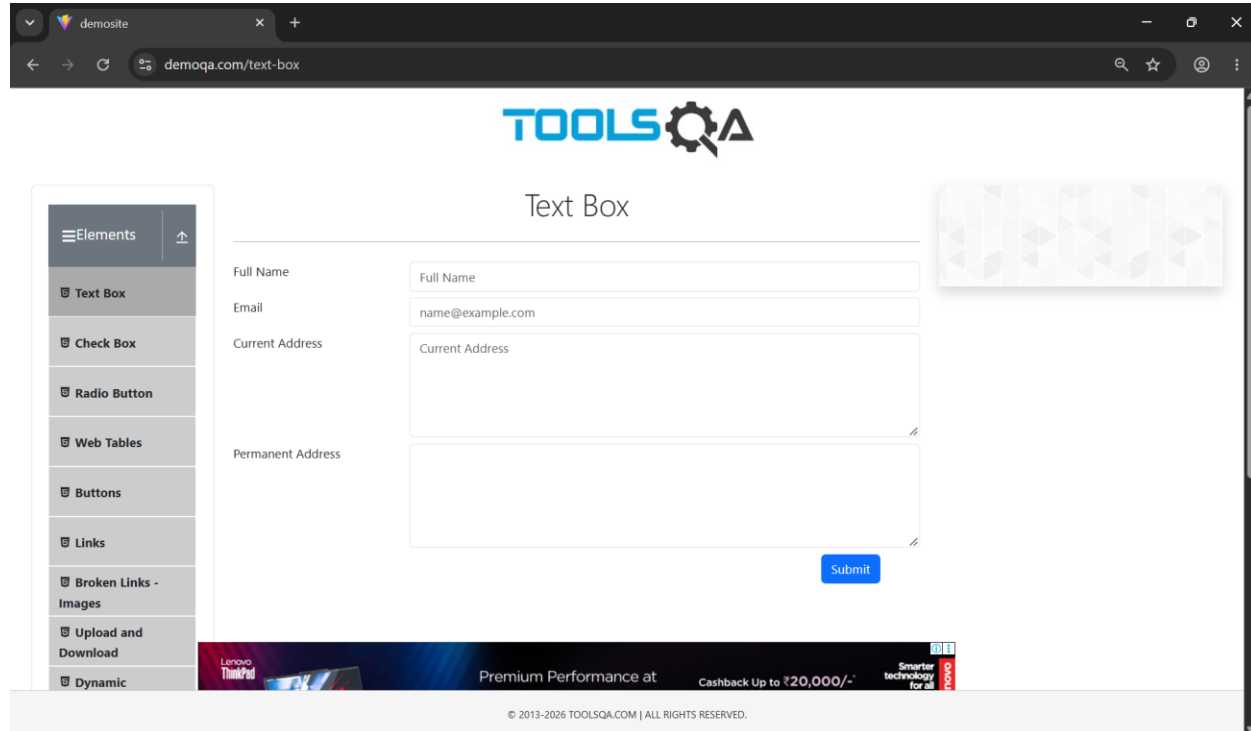
@when("I submit the form")
def submit_form(context):
    driver = context.driver
    driver.find_element(
        By.ID, "submit"
    ).click()
    print("Form submitted")

@then("I should see the submitted details")
def verify_details(context):
    driver = context.driver
    output = WebDriverWait(driver, 10).until(
        EC.visibility_of_element_located(
            (By.ID, "output")
        )
    )
    assert "Sristi Saha" in output.text
    assert "sristi@example.com" in output.text
    assert "Kolkata, India" in output.text
    print("PASS: Submitted details verified")
    input(" Press ENTER.")

```

Output

1.1 Initial DemoQA text box page



1.2 Completed form

The screenshot shows a web browser window with the URL `demoqa.com/text-box`. The page features the ToolsQA logo at the top. On the left, there is a sidebar with a list of elements: Elements, Text Box, Check Box, Radio Button, Web Tables, Buttons, Links, Broken Links - Images, Upload and Download, and Dynamic. The main content area is titled 'Text Box' and contains a form with the following fields:

- Full Name: Sristi Saha
- Email: sristi@example.com
- Current Address: Kolkata, India
- Permanent Address: West Bengal, India

A blue 'Submit' button is located at the bottom right of the form. Below the form, there is a banner for 'Premium Performance at' with a cashback offer of 'Up to ₹20,000/-'. The footer of the page states '© 2013-2026 TOOLQA.COM | ALL RIGHTS RESERVED.'

1.3 Submitted form output

The screenshot shows the same web browser window as in the previous image, but now the form has been submitted. The 'Submit' button is no longer visible. Instead, the form fields are now part of a summary output box at the bottom of the form area, displaying the following text:

Name:Sristi Saha
Email:sristi@example.com
Current Address :Kolkata, India
Permanenet Address :West Bengal, India

The footer of the page remains the same: '© 2013-2026 TOOLQA.COM | ALL RIGHTS RESERVED.'

1.4 Behavior terminal result

```
Terminal  Local (2) ×  Local ×  Local (3) ×  +  v

Windows PowerShell
Copyright (C) Microsoft Corporation. All rights reserved.

(.venv) PS C:\Users\Sristi\PycharmProjects\PythonProject\Selenium Lab Assignments\Module 3\Assignment1> behave
USING RUNNER: behave.runner:Runner
Feature: Selenium automation using Python Behave # features/login.feature:1

DevTools listening on ws://127.0.0.1:54385/devtools/browser/eb93a140-6d73-4a02-9ba4-b21c388bcfdd

  Scenario: Submit the DemoQA text box form # features/login.feature:3
    Created TensorFlow Lite XNNPACK delegate for CPU.
    Given I open the DemoQA text box page # features/steps/selenium_steps.py:7 32.012s
    And I submit the form # features/steps/selenium_steps.py:43 0.171ss
    Then I should see the submitted details # features/steps/selenium_steps.py:54 8.048s

1 feature passed, 0 failed, 0 skipped
1 scenario passed, 0 failed, 0 skipped
4 steps passed, 0 failed, 0 skipped
Took 0min 54.785s
```

Result and Observation

The Python Behave framework was executed successfully with Selenium WebDriver. The test opened the DemoQA Text Box page, entered the form values, submitted the form and verified the displayed output using assertions. The recorded execution result was 1 feature passed, 1 scenario passed and 4 steps passed, with no failures or skipped steps. Chrome emitted some background-service diagnostics, but these did not cause the Behave scenario to fail.

Conclusion

The experiment demonstrated the setup and execution of an end-to-end Selenium automation scenario using Python and Behave. It provided practical experience with Gherkin, step definitions, browser control, assertions and screenshot checkpoints.

Experiment 2: Use of Test Data-Driven Automation in Python Behave Framework

Module	Python BDD RESTful Automations
Experiment Title	Use of Test Data-Driven Automation in Python Behave Framework
API Automation	https://jsonplaceholder.typicode.com

Problem Statement

Develop a data-driven API automation framework using Python and Behave. The framework should execute the same API test with multiple input values, send HTTP GET requests, parse JSON responses and validate response status codes and returned user information.

Objective

To adapt the Python BDD framework for API automation and demonstrate parameterized testing through a Gherkin Scenario Outline and Examples table. The test uses JSONPlaceholder as a public sample REST API.

Tools, Software, and Concepts Used

- Python 3
- Behave BDD framework and Gherkin Scenario Outline / Examples
- Requests library for HTTP GET calls
- JSON response parsing and assertions
- HTTP status codes, endpoint handling, timeout and exception handling
- JSONPlaceholder sample REST API

Implementation Steps

1. Install Behave and Requests using pip.
2. Create features/api.feature with a Scenario Outline and a table of user IDs.
3. Create features/environment.py to define the API base URL and scenario context.
4. Implement the step definitions in features/steps/api_steps.py.

Source Code —

features/api.feature

Feature: Data-driven API testing using Behave

Scenario Outline: Retrieve user details using API

```
Given the API endpoint is available
When I send a GET request for user <user_id>
Then the response status should be <status>
And the response should contain the expected user ID <user_id>
```

Examples:

	user_id	status
	1	200
	2	200
	3	200
	4	200
	5	200

features/environment.py

```
def before_scenario(context, scenario):
    context.base_url = (
        "https://jsonplaceholder.typicode.com"
    )
    context.response = None
```

features/steps/api_steps.py

```
from behave import given, when, then
import requests
```

```
@given("the API endpoint is available")
```

```
def check_endpoint(context):
    response = requests.get(
        context.base_url + "/users/1",
        timeout=15
    )
    assert response.status_code == 200
    print("API endpoint is accessible")
    print("Status code:", response.status_code)
    input( " Press ENTER." )
```

```
@when("I send a GET request for user {user_id:d}")
```

```
def get_user(context, user_id):
    url = f"{context.base_url}/users/{user_id}"
    context.response = requests.get(
        url,
        timeout=15
    )
    print("\nRequest URL:", url)
    print("HTTP Method: GET")
    print("Response Status:", context.response.status_code)
    print("Response Body:", context.response.text)
    input(Press ENTER.)
)
```

```
@then("the response status should be {status:d}")
```

```
def verify_status(context, status):
    assert context.response.status_code == status
    print(
        f"PASS: Expected status {status}, "
        f"received {context.response.status_code}"
    )
```

```

@then("the response should contain the expected user ID {user_id}")
def verify_user_id(context, user_id):
    data = context.response.json()
    assert data["id"] == user_id
    assert "name" in data
    assert "email" in data
    print("PASS: User ID verified:", data["id"])
    print("User Name:", data["name"])
    print("User Email:", data["email"])
    input(Press ENTER."
)

```

Output

2.1 Terminal result

```

Terminal Local x + v
(.venv) PS C:\Users\Sristi\PycharmProjects\PythonProject\Selenium Lab Assignments\Module 3\Assignment2> behave
USING RUNNER: behave.runner:Runner
Feature: Data-driven API testing using Behave # features/api.feature:1

  Scenario Outline: Retrieve user details using API -- @1.1 # features/api.feature:11
    Given the API endpoint is available # features/steps/api_steps.py:5 32.530s
    Then the response status should be 200 # features/steps/api_steps.py:43 0.000ss
    And the response should contain the expected user ID 1 # features/steps/api_steps.py:53 0.506s

  Scenario Outline: Retrieve user details using API -- @1.2 # features/api.feature:12
    Given the API endpoint is available # features/steps/api_steps.py:5 1.018s
    Then the response status should be 200 # features/steps/api_steps.py:43 0.001s
    And the response should contain the expected user ID 2 # features/steps/api_steps.py:53 0.378s

  Scenario Outline: Retrieve user details using API -- @1.3 # features/api.feature:13
    Given the API endpoint is available # features/steps/api_steps.py:5 1.606s
    Then the response status should be 200 # features/steps/api_steps.py:43 0.001s
    And the response should contain the expected user ID 3 # features/steps/api_steps.py:53 1.735s

  Scenario Outline: Retrieve user details using API -- @1.4 # features/api.feature:14
    Given the API endpoint is available # features/steps/api_steps.py:5 1.094s
    Then the response status should be 200 # features/steps/api_steps.py:43 0.000s
    And the response should contain the expected user ID 4 # features/steps/api_steps.py:53 1.072s

  Scenario Outline: Retrieve user details using API -- @1.5 # features/api.feature:15
    Given the API endpoint is available # features/steps/api_steps.py:5 0.944s
    Then the response status should be 200 # features/steps/api_steps.py:43 0.000s
    And the response should contain the expected user ID 5 # features/steps/api_steps.py:53 0.708s

1 feature passed, 0 failed, 0 skipped
5 scenarios passed, 0 failed, 0 skipped
20 steps passed, 0 failed, 0 skipped
Took 1min 46.602s

```

Result and Observation

The implementation uses a Gherkin Scenario Outline to run a parameterized GET test against multiple user IDs. The Requests library sends HTTP calls, and the step definitions validate status codes and JSON fields. A request timeout and exception handling are included to make connection failures easier to diagnose. The final result should be recorded from the user's actual run; the public test API requires internet access and may not be available in all environments

Conclusion

The experiment demonstrates data-driven API testing using Python, Behave, Gherkin and Requests. Parameterized examples reduce duplicated test logic and make it straightforward to extend the suite with additional test data. The optional POST script illustrates a separate HTTP method but is supplementary to the core assignment.

Experiment 3: Selenium Page Object Model in the Python Behave Framework

Module	Python BDD RESTful Automations
Experiment Title	Selenium Page Object Model in the Python Behave Framework
Web Automation	https://www.saucedemo.com

Problem Statement

Implement and organize web application automation using the Page Object Model design pattern within a Python Behave framework. Page locators and browser interaction methods should be encapsulated in a page class, while the Gherkin scenarios and step definitions describe and validate user behavior.

Objective

To debug, structure and execute maintainable Selenium browser automation using Behave and Gherkin, applying reusable page methods to test both successful and unsuccessful login flows.

Tools, Software, and Concepts Used

- Python 3
- Behave BDD framework and Gherkin feature files
- Selenium WebDriver 4 and Google Chrome
- Page Object Model (POM), page classes and locator tuples
- WebDriverWait and expected_conditions
- Assertions, positive testing, negative testing and browser lifecycle hooks

Implementation Steps

1. Create the Assignment3 project with features, features/steps and pages directories.
2. Implement LoginPage in pages/login_page.py, defining locators and reusable page interaction methods.
3. Create features/login.feature with valid-credential and invalid-credential scenarios.
4. Configure Chrome initialization and teardown in features/environment.py.
5. Implement the Gherkin step definitions in features/steps/login_steps.py and import LoginPage.
6. Run behave from the Assignment3 project root and inspect both browser flows.

Source Code —

pages/login_page.py

```
from selenium.webdriver.common.by import By
from selenium.webdriver.support.ui import WebDriverWait
from selenium.webdriver.support import expected_conditions as EC
```

```
class LoginPage:
    URL = "https://www.saucedemo.com/"

    USERNAME = (By.ID, "user-name")
```

```
PASSWORD = (By.ID, "password")
LOGIN_BUTTON = (By.ID, "login-button")
ERROR_MESSAGE = (
    By.CSS_SELECTOR,
    "[data-test='error']" )
```

```
INVENTORY_CONTAINER = (
    By.ID,
    "inventory_container" )
```

```
def __init__(self, driver):
    self.driver = driver
    self.wait = WebDriverWait(driver, 15)
```

```
def open(self):
    self.driver.get(self.URL)
```

```
def enter_username(self, username):
    element = self.wait.until(
        EC.visibility_of_element_located(
            self.USERNAME )
    )
    element.clear()
    element.send_keys(username)
```

```
def enter_password(self, password):
    element = self.wait.until(
        EC.visibility_of_element_located(
            self.PASSWORD )
    )
    element.clear()
    element.send_keys(password)
```

```
def click_login(self):
    self.wait.until(
        EC.element_to_be_clickable(
            self.LOGIN_BUTTON )
    ).click()
```

```
def get_error_message(self):
    return self.wait.until(
        EC.visibility_of_element_located(
            self.ERROR_MESSAGE )
    ).text
```

```
def is_inventory_displayed(self):
    return self.wait.until(
        EC.visibility_of_element_located(
            self.INVENTORY_CONTAINER
        )
    ).is_displayed()
```

features/login.feature

Feature: Login using Page Object Model

Scenario: Login with valid credentials

Given I open the Sauce Demo login page

When I enter username "standard_user"

And I enter password "secret_sauce"

And I click the login button

Then I should be redirected to the inventory page

Scenario: Login with invalid credentials

Given I open the Sauce Demo login page

When I enter username "invalid_user"

And I enter password "wrong_password"

And I click the login button

Then I should see the login error message

features/environment.py

```
from selenium import webdriver
from selenium.webdriver.chrome.options import Options
```

```
def before_scenario(context, scenario):
    options = Options()
    options.add_argument("--start-maximized")
    context.driver = webdriver.Chrome(
        options=options)
```

```
def after_scenario(context, scenario):
    if hasattr(context, "driver"):
        input(
            "\nPress ENTER to close the browser "
            "and continue...")
        context.driver.quit()
```

features/steps/login_steps.py

```
from behave import given, when, then
from pages.login_page import LoginPage
```

```
@given("I open the Sauce Demo login page")
def open_login_page(context):
    context.login_page = LoginPage( context.driver )
    context.login_page.open()
    print("Sauce Demo login page opened")
    input("Press ENTER.")
```

```
@when("I enter username \"{username}\"")
def enter_username(context, username):
    context.login_page.enter_username(
        username)
    print("Username entered:", username)
```

```

@when("I enter password \"{password}\"")
def enter_password(context, password):
    context.login_page.enter_password(
        password)
    print("Password entered")
    input(" Press ENTER.")

@when("I click the login button")
def click_login(context):
    context.login_page.click_login()
    print("Login button clicked")

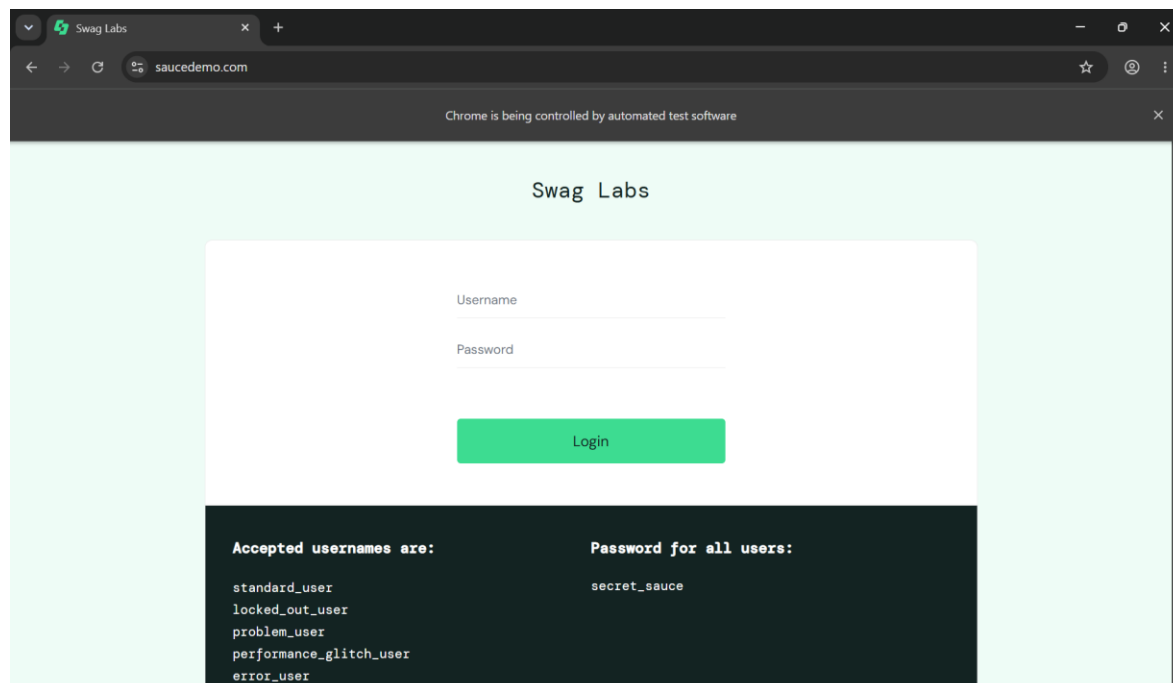
@then("I should be redirected to the inventory page")
def verify_successful_login(context):
    assert context.login_page.is_inventory_displayed()
    print("PASS: Inventory page displayed")
    print("Current URL:", context.driver.current_url)
    input(" Press ENTER.")

@then("I should see the login error message")
def verify_login_error(context):
    error = context.login_page.get_error_message()
    assert "Username and password do not match" in error
    print("PASS: Invalid login correctly rejected")
    print("Error message:", error)
    input("Press ENTER.")

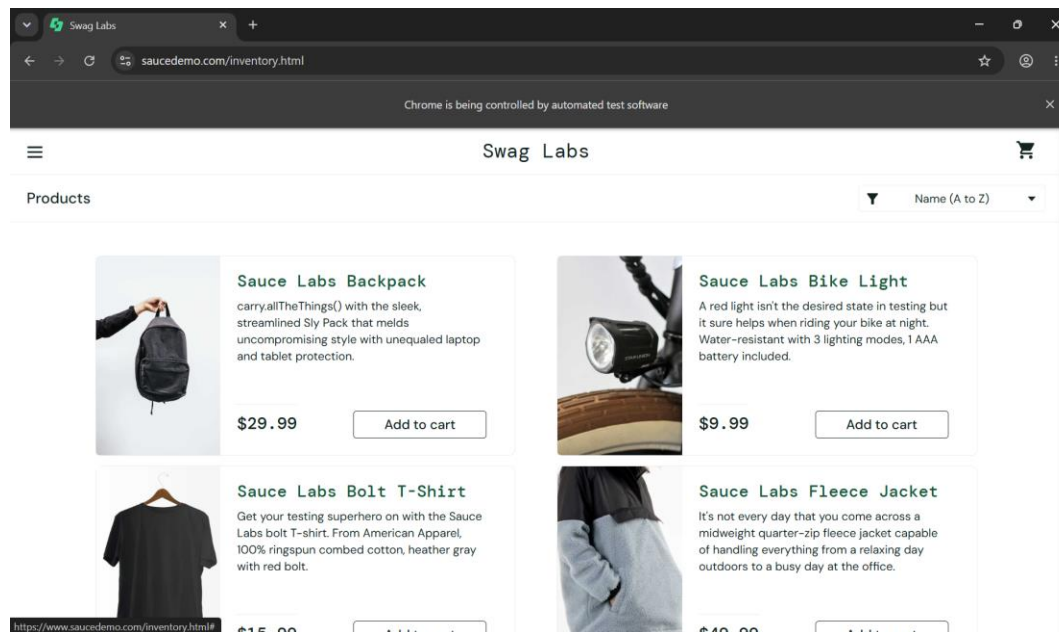
```

Output

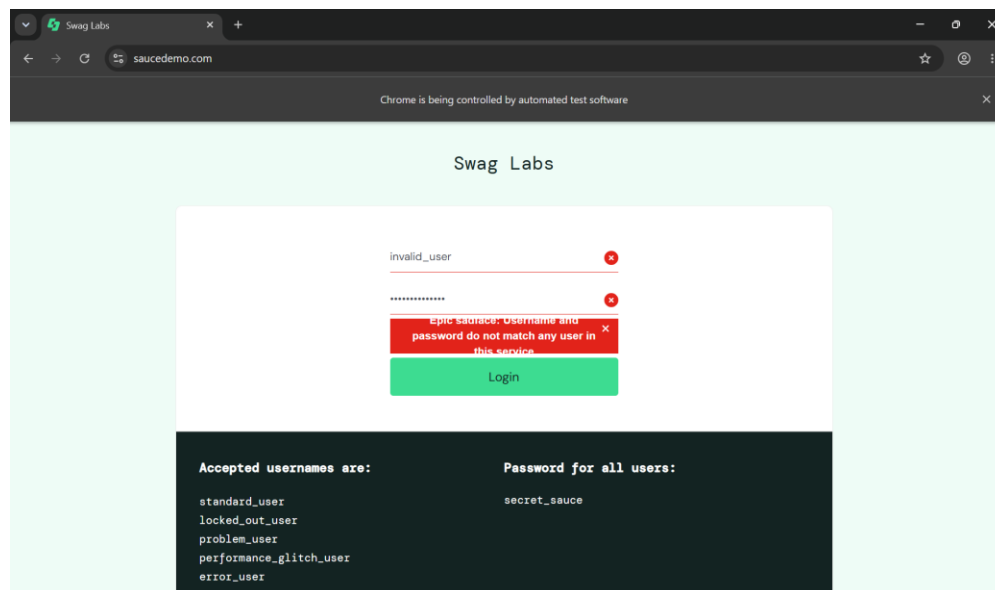
3.1 Sauce Demo login page



3.2 Successful login – Inventory page



3.3 Invalid login error



3.4 Terminal summary

```
(.venv) PS C:\Users\Sristi\PycharmProjects\PythonProject\Selenium Lab Assignments\Module 3\Assignment3> behave
USING RUNNER: behave.runner.Runner
Feature: Login using Page Object Model # features/login.feature:1

DevTools listening on ws://127.0.0.1:51833/devtools/browser/058602bc-95a0-4d76-bacc-9727c49a7068

Scenario: Login with valid credentials # features/login.feature:3
  Given I open the Sauce Demo login page # features/steps/login_steps.py:5
[31948:2176:0927/163356.337:ERROR:google_apis\gcm\engine\registration_request.cc:291] Registration response error message: PHONE_REGISTRATION_ERROR
[31948:2176:0927/163356.363:ERROR:google_apis\gcm\engine\registration_request.cc:291] Registration response error message: PHONE_REGISTRATION_ERROR
[31948:2176:0927/163356.364:ERROR:google_apis\gcm\engine\registration_request.cc:291] Registration response error message: PHONE_REGISTRATION_ERROR
[31948:2176:0927/163356.436:ERROR:google_apis\gcm\engine\mcs_client.cc:702] Error code: 401 Error message: Authentication Failed: wrong_secret
  When I enter username "standard_user" # features/steps/login_steps.py:21 0.284s GCM, resetting connection.
  And I enter password "secret_sauce" # features/steps/login_steps.py:30
```

```

And I click the login button # features/steps/login_steps.py:44 0.253s son response error message: DEPRECATED_ENDPOINT
Then I should be redirected to the inventory page # features/steps/login_steps.py:51
Then I should be redirected to the inventory page # features/steps/login_steps.py:51 8.312s

DevTools listening on ws://127.0.0.1:59621/devtools/browser/a8594027-9697-461c-9954-f7ff2ee98600

Scenario: Login with invalid credentials # features/login.feature:10
  Given I open the Sauce Demo login page # features/steps/login_steps.py:5
[28668:6640:0927/163446.068:ERROR:google_apis\gcm\engine\registration_request.cc:291] Registration response error message: PHONE_REGISTRATION_ERROR
[28668:6640:0927/163446.078:ERROR:google_apis\gcm\engine\registration_request.cc:291] Registration response error message: PHONE_REGISTRATION_ERROR
[28668:6640:0927/163446.090:ERROR:google_apis\gcm\engine\registration_request.cc:291] Registration response error message: PHONE_REGISTRATION_ERROR
[28668:6640:0927/163446.521:ERROR:google_apis\gcm\engine\mcs_client.cc:702] Error code: 401 Error message: Authentication Failed: wrong_secret
[28668:6640:0927/163446.521:ERROR:google_apis\gcm\engine\mcs_client.cc:704] Failed to log in to GCM, resetting connection.
  Given I open the Sauce Demo login page # features/steps/login_steps.py:5 8.923s
  When I enter username "invalid_user" # features/steps/login_steps.py:21 0.306s
  And I enter password "wrong_password" # features/steps/login_steps.py:30
  And I enter password "wrong_password" # features/steps/login_steps.py:30 5.182s
  And I click the login button # features/steps/login_steps.py:44 0.339s
  Then I should see the login error message # features/steps/login_steps.py:65
Created TensorFlow Lite XNNPACK delegate for CPU.
  Then I should see the login error message # features/steps/login_steps.py:65 11.124s

1 feature passed, 0 failed, 0 skipped
2 scenarios passed, 0 failed, 0 skipped
10 steps passed, 0 failed, 0 skipped
Took 1min 3.112s

```

Result and Observation

The Page Object Model was implemented through a LoginPage class containing page locators and reusable interaction methods. Behave step definitions invoked these methods while Gherkin scenarios described the user flows. The valid-login scenario verified that the inventory page appeared, and the invalid-login scenario verified that the expected error message was displayed. The actual terminal output reported 1 feature passed, 2 scenarios passed and 10 steps passed, with no failures or skips. The initial module import error was resolved before the successful run.

Conclusion

The experiment demonstrated how the Page Object Model can organize Selenium automation within a Behave BDD framework. Encapsulating locators and UI operations in a page class keeps step definitions focused on behavior and validation, improving readability, reuse and maintainability.

Overall Module Result

The three practical assignments cover the core workflow of Python BDD RESTful automation: establishing a Behave-based browser test, applying data-driven techniques to API testing, and structuring Selenium web tests using the Page Object Model.

Overall Observation

- Behave connects readable Gherkin scenarios with executable Python step definitions.
- Selenium WebDriver supports browser navigation, form interactions and UI assertions.
- Requests enables HTTP API calls and JSON response validation without a browser.
- Scenario Outline and Examples support parameterized, data-driven test execution.
- The Page Object Model separates page-specific locators and actions from scenario logic.
- Explicit waits improve synchronization with dynamically rendered web elements.
- Terminal summaries and labelled screenshots provide evidence of test execution.

Overall Conclusion

The module provides practical experience in developing structured Python automation with BDD principles. Assignment 1 demonstrated a browser-based end-to-end form workflow; Assignment 2 applies Behave to parameterized API response testing; and Assignment 3 organizes Selenium tests using the Page Object Model. Together, these exercises establish a foundation for reusable, readable and maintainable automation frameworks.